LocalMind: A Local-First Agentic Knowledge Copilot With Persistent Project Memory and a Custom MCP Server

Trijal Burman, Tiya Singh, Gayatri Rajesh Apotikar, and Nimita Jestin
Symbiosis Institute of Technology

***Abstract*—Persistent project context is difficult to recover across scattered local documents and conversations without transmitting personal knowledge to hosted services. This paper presents LocalMind, an implemented copilot combining local inference, hybrid retrieval, relational project memory, and a custom Model Context Protocol (MCP) server. A bounded agent reads approved-folder evidence through actual MCP subprocess sessions. SQLite retains extracted text, vectors, notes, and conversations. Qwen3 4B and EmbeddingGemma run on an RTX 5050 laptop with 16 GB nominal RAM. On LM-Synth v1, a reproducible 39-document corpus, lexical and hybrid retrieval cover all 96 answerable questions within five results. The frozen baseline nevertheless matches 0/24 complete answers because context clipping discards evidence. A historical post hoc adapter identifies the mechanism. A subsequent production repair fits actual serialized UTF-8 cost; a separate replay retains initial references in 64/64 requests, respects the configured input budget, and yields 7/24 single-pass and 3/24 agent all-fact value matches with no request errors. Both replays reuse inspected questions and are not held-out confirmation; passage-prefix losses remain. Historical summaries visit every section yet retain only 10/24 requested values and have not been rerun after repair. Current Linux verification passes 62 tests; the repaired Windows copy passes 99 on Linux with six native checks skipped. Earlier native Windows Server jobs passed 86 tests per interpreter but do not validate the new repair on Windows. These alias-based results demonstrate local integration and a verified context-delivery repair, not general semantic accuracy. No paid inference API is used; broader accuracy, security, and Windows laptop GPU performance remain unverified.**

***Index Terms*—local-first AI, retrieval-augmented generation, Model Context Protocol, persistent memory, on-device inference, knowledge management**

I. Introduction

Knowledge work produces heterogeneous evidence: project notes, source files, reports, decisions, and conversational explanations. Relevant information is often available on a user's disk but absent from the current assistant context. Repeatedly uploading or describing that information increases effort, while hosted inference introduces a separate data-processing boundary. Keeping a long conversation open does not provide a durable, searchable project memory, and increasing a model's context window does not ensure that every relevant fact is used.

Local-first software emphasizes ownership and offline availability [1]. Retrieval-augmented generation (RAG) makes external evidence available to a generator [2], while persistent-memory research explores continuity beyond one context window [3], [4]. These established directions motivate a practical question: can a modest laptop combine document retrieval, explicit project memory, and interoperable tool use in one locally operated system with inspectable evidence?

We address that question through LocalMind, an implemented web application backed by a local database and local models. A custom MCP server separates knowledge access from the conversational host [5]. The application allows a user to approve folders on an HDD or SSD, index supported text, inspect complete extracted contents, ask grounded questions, save memories, and draft project plans. It does not automatically scan every drive or execute generated software. Initial installation requires downloads; subsequent document processing and inference can operate without a hosted API.

This paper makes three contributions. First, it describes the concrete integration of local inference, hybrid retrieval, relational project memory, and actual MCP subprocess communication, including the controls that bound the agent's capabilities. Second, it provides an explicit distinction among complete extraction, retrieved evidence, and generated summary coverage. Third, it supplies a frozen synthetic corpus, executable evaluation, raw outputs, and laptop measurements, including a separately recorded production-repair replay that leaves the original results unchanged. None of RAG, rank fusion, memory, or MCP is proposed as a new algorithm.

Four research questions structure the evaluation: *RQ1*, how do lexical, dense, and fused retrieval cover labeled evidence? *RQ2*, how does the bounded MCP agent compare with single-pass local RAG on answer-value matching and missing information? *RQ3*, does processing every extracted section retain the requested facts in the final summary? *RQ4*, what runtime costs and functional guarantees are supported on the measured platforms? The answers are conditional on the stated corpus, implementation revisions, and hardware.

II. Related Work

A. Retrieval and Grounded Generation

RAG combines retrieved nonparametric knowledge with generation [2]. Dense Passage Retrieval uses learned dual encoders for passage matching [6]; Sentence-BERT demonstrates efficient independent representations for similarity computations [7]. BM25 remains a strong lexical foundation with term-frequency saturation and length normalization [8]. Dense representations can help with paraphrases, while lexical

retrieval preserves identifiers and exact terminology. Their relative value depends on the corpus and query distribution.

Reciprocal rank fusion (RRF) combines ranked lists without requiring their raw scores to share a scale [9]. LocalMind applies this established technique to SQLite FTS5 and local cosine retrieval. It does not train the retriever jointly with the generator, fine-tune embeddings, or implement an approximate nearest-neighbor index. Accordingly, the present comparison concerns the deployed pipeline rather than the performance of the original RAG or DPR systems.

B. Tool Use and Persistent Memory

ReAct interleaves information gathering with model-selected actions [10]. Toolformer studies learning when and how to invoke APIs [11]. LocalMind uses an already trained model and a bounded runtime loop; it neither reproduces Toolformer's training procedure nor exposes internal reasoning as an experimental variable.

MemGPT investigates hierarchical memory and virtual context management [3]. MemoryBank targets persistent user-oriented memory [4], and LongMem incorporates a learned memory architecture [12]. Generative Agents uses memory and reflection for simulated behavior [13]; Reflexion studies feedback retained across trials [14]. In contrast, LocalMind stores explicit notes, projects, and conversations in ordinary relational records. It has no learned memory consolidation, forgetting policy, autonomous reflection, or automatic preference extraction. Explicit user control over writes is a deliberate capability restriction.

C. Context, Ownership, and Interoperability

Long contexts do not remove selection and position effects: Lost in the Middle reports that relevant information can be used unevenly across input positions [15]. This motivates separating evidence retrieval from full-document processing and measuring the latter's retained facts. A section trace establishes that text entered a processing stage; it cannot establish semantic completeness.

The broader local-first vision includes collaboration and replication [1]. LocalMind implements single-laptop ownership and offline operation, not collaborative synchronization or conflict-free replicated data types. MCP supplies a common host–client–server interface for tools and resources [5]; interoperability does not itself enforce authorization. Indirect prompt injection further motivates treating retrieved content as untrusted [16]. Local inference changes where processing occurs, while runtime capabilities determine what a compromised response can do.

D. Evaluation Foundations

BEIR emphasizes heterogeneous retrieval evaluation and the importance of lexical baselines [17]; MTEB evaluates embeddings across tasks [18]. RAGAs separates retrieval, faithfulness, and answer quality [19], while citation-oriented generation research evaluates whether references support claims [20]. We retain those distinctions. Public model scores are not transferred to this implementation, and quantitative scores use no model judge. LM-Synth supports transparent value-level diagnostics, with a narrower scope than those public benchmarks.

III. Requirements and Threat Model

The target deployment is one trusted user on a laptop with a Ryzen 7 250, RTX 5050, and 16 GB nominal RAM. The design requires persistent project context, source-backed answers, complete extracted-text access, bounded inference, and continued lexical operation when models are unavailable. The system must not require a paid API, hosted vector service, or remote database. "No paid inference API" does not mean zero hardware, electricity, storage, bandwidth, or maintenance cost.

The privacy boundary contains the local browser, API, SQLite database, approved document folders, MCP subprocess, and loopback model server. Application inference endpoints accept HTTP loopback hosts only, reject remote-model metadata, and bypass inherited HTTP proxies. Bundled UI assets avoid a CDN dependency. The model runtime is configured with cloud access disabled [21]. These controls describe application data flow; they are not a formal proof that the operating system or all installed software is offline.

We consider accidental overbroad folder selection, stale documents, indirect instructions in retrieved text, malformed tool arguments, and unbounded tool execution. We do not assume resistance to a compromised OS account, malicious kernel, physical disk access, or a hostile application with the same file permissions. SQLite is not encrypted by the application. Known-secret filename exclusions are heuristics; an ordinary note can still contain sensitive information. A hosted external MCP client can transmit returned data outside this boundary.

The Linux scanner rejects protected system roots and symbolic-link traversal, but it does not identify the physical backing type of every mounted filesystem. An explicitly approved network-backed mount is therefore outside a strict physical-local-disk guarantee. The Windows edition additionally restricts accepted paths to supported local fixed/removable drives and rejects UNC/network paths, junctions, and reparse-backed sources. This platform difference is preserved rather than concealed by a universal privacy claim.

IV. System Architecture

A. Components and Persistent State

Fig. 1 shows the deployed components. A bundled browser UI communicates with FastAPI, which coordinates indexing, conversations, memories, and projects. Ollama provides the chat model and text embeddings over loopback. The official Python MCP SDK launches and negotiates the separate memory service; the protocol boundary is exercised by actual subprocess tests, not simulated function names [22].

SQLite stores approved roots, document metadata and full extracted text, offset-preserving chunks, vector BLOBs, explicit memories, projects, sessions, and messages. FTS5

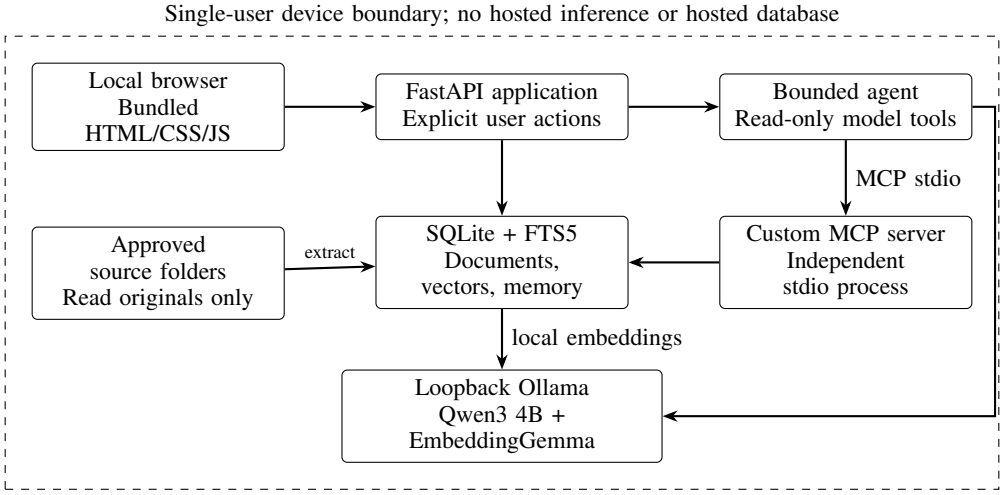


Fig. 1. Implemented data flow. Original files are read through approved roots. The API and MCP server share relational storage. The model receives bounded evidence through the agent; it receives no source-file write or shell tool. The boundary assumes trusted local software and a locally operated MCP client.

indexes chunk content and conversation messages [23]. Short-lived connections, foreign keys, a busy timeout, and write-ahead logging support normal reads while another process writes. The API and MCP client share the same configured data directory.

Explicit memory is a relational note with a kind and optional project identifier. Memory lookup uses bounded substring filtering; conversation lookup uses FTS5. Neither is a knowledge graph or a semantic vector index over every past chat. The agent initially selects bounded global and project notes, while project filtering limits relevant conversation tools. Persistence and retrieval are separate: saving a note does not guarantee that it will fit every future prompt.

B. Approved Ingestion and Complete Text

The user selects existing folders; the app rejects overlapping roots, its own data directory, protected roots, and unsupported links. Scanning skips hidden items, known credential names, dependencies, and unsupported formats. Text and common code files are decoded locally. PDF text is extracted with page boundaries, and DOCX text is extracted from relevant XML parts, including tables and available notes. No OCR, image interpretation, or visual-layout reproduction is provided.

Table I lists implemented bounds. Opening each Linux path component without following links helps prevent replacement through a symbolic link. Windows uses native handles and reparse-point checks. Extracted text is stored in full as a snapshot, with exact character offsets in each chunk. PDF chunks stay within a page. "Complete contents" therefore means the complete successful text extraction of a supported indexed file, not the original formatting or every byte on a drive.

Incremental scans compare modification time and size; changed documents replace their extraction and chunks, while unchanged documents can receive missing embeddings. Removed rows are pruned only after complete enumeration. Retrieval and complete-text reads reject sources whose recorded metadata is stale. This is metadata-based freshness, not cryptographic verification: changes preserving both size and timestamp can evade it. Rescans are explicit; there is no background watcher.

TABLE I
IMPLEMENTED RESOURCE AND EXECUTION BOUNDS

Parameter	Bound or default
Original source size	20 MiB per file
Scan candidates	20,000 supported files
Extracted text / PDF	5,000,000 characters / 2,000 pages
Chunk / overlap	1,800 / 200 characters
Embedding batch	16 chunk texts
Stored vector	Float32 BLOB, model tag, dimension
MCP document page	Up to 50,000 characters
Chat context	8,192 tokens configured
Agent loop	4 model/tool rounds; at most 6 additional tool executions
Final synthesis	At most one additional call, tools disabled
Summary section	Up to 4,000 characters at the default context

C. Model Configuration

We use Qwen3 4B Instruct 2507 in Q4_K_M quantization. The installed tag is `qwen3:4b-instruct-2507-q4_K_M`. Qwen3's family report provides model background, while the artifact tag and digest identify the actual experiment [24]. The embedding artifact is `embeddinggemma:latest`, a roughly 308M-parameter BF16 representation model with 768 output dimensions and a 2,048-token input context [25]. That context is independent of the chat model's 8,192-token setting.

The application uses temperature 0.25, a generation cap of 1,536 tokens at the default context, ten-minute keep-alives, and no hosted fallback. The default production seed is not fixed.

The experiment retains these settings rather than selecting outputs from repeated attempts. Exact model digests, package versions, corpus hashes, and run identities accompany the results.

V. Retrieval and Memory Access

A. Chunking and Lexical Ranking

For document text x, a chunk $c_i = x[a_i : b_i]$ preserves exact offsets. Chunking targets 1,800 characters and prefers a newline or sentence delimiter in the latter half of the candidate window; adjacent chunks overlap by 200 characters. These are character limits, not token limits. The overlap reduces boundary losses without making extraction or model input unlimited.

FTS5 searches text, title, and path using BM25 field weights 1.0, 2.0, and 0.3, respectively. The application tokenizes at most 24 distinct query words and joins them with OR. Generic BM25 combines inverse document frequency, saturated term frequency, and length normalization [8]; SQLite negates its BM25 score, so lower values rank first [23]. Identifiers and project names can therefore dominate this small corpus, an effect considered when interpreting dense retrieval.

B. Dense Ranking and Fusion

Let $q \in \mathbb{R}^{768}$ be a query embedding and v_i a compatible stored chunk vector. Dense similarity is

$$s(q, v_i) = \frac{q^\top v_i}{\|q\|_2 \|v_i\|_2}. \tag{1}$$

The implementation checks finite, nonzero vectors, compatible dimensions, and matching model labels. Only positive cosine candidates enter the semantic ranking. It scans vectors in batches of 128 rows and retains a bounded heap rather than loading a large Python list of every vector.

For lexical and dense ranks $r_L(c)$ and $r_D(c)$, absent terms contributing zero, fusion is

$$R(c) = \frac{\mathbb{1}[c \in L]}{60 + r_L(c)} + \frac{\mathbb{1}[c \in D]}{60 + r_D(c)}. \tag{2}$$

The constant 60 follows the established RRF method [9]. Candidate depth is $\min(160, \max(40, 4\ell))$, where ℓ is the store-level requested limit. The library overfetches before checking source freshness. Ties use deterministic chunk identifiers. If embeddings are unavailable or fail, lexical search remains usable.

This is exhaustive cosine retrieval with $O(Nd)$ similarity work per query for N compatible chunks of dimension d, plus bounded candidate retention. Streaming bounds working memory for ranking; it does not establish constant memory for extraction, model inference, or the full application. No large-archive scalability result is claimed.

C. MCP Contract and Capability Boundary

The server exposes knowledge search, paginated document reading, memory recall, project/document listing, conversation search, and explicit memory/project mutations. Tools carry protocol annotations and validated argument bounds. The host negotiates an actual SDK session, discovers schemas, and exchanges structured results over stdio. Stdout is reserved for protocol messages.

The built-in model sees only six read tools: knowledge search, document read, recall, document listing, project listing, and conversation search. A requested tool outside that set is rejected. Server annotations are not an authorization mechanism; the host's allowlist enforces this narrower model boundary. External clients can access the server's write tools and must impose their own authorization policy. No server tool executes a shell or reads an arbitrary caller-supplied filesystem path.

VI. Bounded Agent and Full-Document Processing

A. Conversation Workflow

Before inference, the agent parses an explicit current-user memory command such as `/remember`. That path writes a note directly; retrieved text cannot invoke it. For ordinary questions, the agent initializes MCP, requests five source passages, and recalls saved context. Recent history, selected memories, project metadata, and numbered passages are marked as reference data. The system prompt instructs the model to distinguish evidence from inference and admit missing information.

The model can request additional read operations over at most four rounds. A per-turn cap permits at most six additional tool executions, excluding the initial search and recall. A final tool-free synthesis call bounds completion. Tool errors are returned as data and recorded in the trace. This is a constrained agentic workflow inspired by tool-using systems [10], not an unrestricted autonomous agent.

The current context fitter reserves generation and framing space, pins the safety instruction and latest user request, and prioritizes saved context and recent tool outputs. It uses a conservative UTF-8 byte budget rather than exact tokenizer counts. A binary search selects the longest whole-character prefix whose actual serialized message cost fits, including JSON escaping, metadata, and a shortening marker. Tool schemas are counted and can be omitted if they do not fit; tool-call metadata is kept whole. Small remaining budgets are checked exactly rather than rejected by obsolete clipping thresholds. The frozen baseline used approximate clipping that could discard an entire evidence block. Section VIII distinguishes that historical failure, the initial evaluation-only diagnostic, and the subsequent local production repair. A retained block can still omit facts through earlier passage-prefix selection.

If the chat model is not ready, the app returns a labeled extractive mode with matching excerpts. A document-specific request without a ready model returns a preview and a

complete-text reading option, not an alleged AI summary. An installed but failing inference server can still produce a surfaced error; the presence check is not a guarantee of successful generation.

B. Complete-Document Summarization

A document-specific request follows MCP pagination until the complete extraction is read. The agent divides text into sections of

$$h = \max(150, \min(4000, C - 3200)) \text{ characters}, \quad (3)$$

where C is the configured chat context. Each section is summarized, then partial summaries are recursively merged. At the default configuration, $h = 4000$; merge input truncates each partial to at most $h/2$ characters, and merge outputs are similarly bounded to ensure convergence.

This process considers every input section but introduces a compression bottleneck. Facts can be omitted during the map stage or truncated during merging. We therefore measure two separate quantities: section-processing coverage and final requested-fact retention. Neither a trace of every section nor a successful full-text transport test establishes perfect summarization. This distinction also addresses context-position concerns raised in prior work [15].

C. Project Planning

A project stores a name, description, and linked notes. A user-triggered plan retrieves indexed evidence through MCP and asks the local model for goals, milestones, architecture, tasks, acceptance criteria, and next steps. Sources found during retrieval are appended as local references, and the result is saved as the project's `PLAN.md`. The implementation drafts a plan; it does not scaffold, compile, or execute arbitrary project code. Appended references improve traceability but do not automatically validate every proposed task or date.

VII. Experimental Method

A. Versions, Hardware, and Execution

The frozen baseline uses Linux commit `ddbbbb7`, preserved separately from the Windows edition [22]. The later production replay uses a locally repaired `copilot/ollama.py`; its SHA-256 identity is recorded in Appendix A. The repair is applied identically to both local platform copies but remains uncommitted and unpublished. Experiments were executed on 17 October 2026 on a Ryzen 7 250 with eight cores and sixteen threads, an RTX 5050 Laptop GPU reporting 8,151 MiB VRAM, and 16 GB nominal RAM (14,768 MiB visible to the OS). The host ran Linux 7.0.0-38-generic, NVIDIA driver 595.99.02, CPython 3.14.4, and Ollama 0.35.0. Dedicated servers at `127.0.0.1:11437` (historical runs) and `127.0.0.1:11438` (production replay) reused downloaded model artifacts and disabled cloud inference. It did not access personal source folders.

The Python environment used NumPy 2.5.3, HTTPX 0.28.1, MCP SDK 1.30.0, FastAPI 0.142.2, and pypdf 6.19.0. Warm-up used a non-answer-bearing readiness prompt. Generation temperature remained 0.25 with no fixed seed. Each frozen primary condition was run once per question, without prompt search, model training, answer selection, or retry of unsuccessful outputs. The initial context-fitter adapter was designed after observing the baseline failure. A later production replay exercises the repaired application directly, with actual MCP calls and no fitter substitution. Both reuse the original 24 answerable and eight absent-value items per method; neither is held-out confirmation. Retrieval and summary measurements remain frozen and are not rerun by this generation-only verification.

B. LM-Synth v1 Corpus

LM-Synth v1 is an authored, deterministic English corpus with seed 20261017. Template and builder-code drafting used AI assistance; the evaluated Qwen3 and EmbeddingGemma models did not generate gold labels or judge outputs. Twelve fictional project families each contain decisions, specifications, and operations records, for 36 ordinary Markdown files. Three additional readiness dossiers support long-document tasks. The corpus has 201,518 Python characters. Ordinary files contain 3,953–4,528 characters; long files contain 16,214, 16,214, and 16,224 characters.

The question set contains 48 lexical, 36 paraphrase, and 12 two-document answerable questions, totaling 96. Each cross-document question requires two gold facts from distinct files; the answerable set therefore contains 108 required fact instances. Eight additional questions request values absent from the sources. Three summary requests each ask for eight facts spread from the beginning to the end of a long dossier. Gold labels specify literal values, permitted rendering aliases, and exact relative document paths. A matching value in the wrong file does not count as retrieved evidence.

Templates and labels were fixed before evaluated-model execution. The dataset, scorer, outputs, and gold-label files remain outside the approved corpus folder; the values themselves occur in the source documents by design. No private documents, real personal conversations, or evaluated-model-generated answers were used to tune the corpus. Repeated project templates and retained project names intentionally make the study easy to audit, but constrain external validity. In particular, a paraphrase question is not a pure test of semantic matching if it still shares a project identifier.

C. Retrieval Conditions and Measures

All answerable questions are evaluated under three conditions: lexical FTS5, cosine-only dense retrieval, and production-style RRF hybrid retrieval. The same indexed text and local vectors are used. Lexical/hybrid ranking uses candidate lists of up to 60 entries, corresponding to the store-level overfetch used for five final passages, and records the top ten results. The dense-only quality control sorts vectorized cosine scores across all 141 chunks and retains ten; its ranking time is not treated as a comparable production service latency. Documents remain unchanged during this comparison, so freshness rejection is not a relevance confound. Every

chunk must have a finite, nonzero, model-compatible 768-dimensional embedding.

For query q with required facts F_q, let G_{qj} be the set of correct-document chunks containing an accepted value for fact j, and let $T_k(q)$ be the first k results. We measure evidence-fact recall as

$$\mathrm{EFR@}\,k = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{|F_q|}\sum_{j\in F_q}\mathbb{1}[T_k(q)\cap G_{qj}\neq\varnothing]. \quad (4)$$

Complete-evidence success (CES@k) requires all facts for a question to be covered. MRR@10 uses the first chunk containing any required fact; it does not require both cross-document facts. These definitions prevent duplicated overlapping chunks from inflating fact recall. Their operational evidence labels use accepted value strings and document paths, not independently annotated entailment or semantic relevance. CES@5 intervals use 2,000 bootstrap resamples of the twelve project clusters, preserving dependence among questions within each project.

D. Generation Conditions and Measures

Generation uses 24 answerable questions, selected by predetermined rotating query-index pairs across projects, giving eight lexical, eight paraphrase, and eight cross-document tasks. All eight absent-value questions are added. The two conditions are single-pass in-process hybrid RAG and the actual bounded MCP agent. They use the same model, context setting, initial five-hit retrieval policy, and a request for at most two sentences. Each asks for the phrase "Not in the documents" when the answer is absent. Hash-determined condition order is balanced 12/12 over answerable items; overall it is 19 agent-first and 13 single-pass-first, so full-set timing is not perfectly counterbalanced.

The single-pass condition omits tool schemas and additional tool calls. The agent additionally incurs MCP initialization, recall, schema context, and possible extra model/tool calls. This is an end-to-end method comparison, not an isolated estimate of MCP protocol overhead or a perfectly controlled ablation of action selection. Responses are not stored as conversation messages, and the evaluation asserts that no prior answer or memory row can leak into later questions.

Fact-value match (FVM) checks case-folded, punctuation-insensitive aliases. All-fact value match (AFVM) requires every requested value in an answer; partial FVM averages fact coverage per question. These are transparent string-based proxies, not human judgments of semantic correctness: a negated, misattributed, or contradictory mention can pass. Citation-index validity and coverage of gold document paths are measured separately; they are not citation entailment or claim-level precision. Absence behavior includes the strict requested-phrase rate. AFVM uncertainty is summarized with project-cluster bootstrap intervals; the small missing-value set is reported with its denominator.

E. Summaries, Resources, and Reproducibility

The three long documents are summarized through the actual document-specific MCP and map/merge path. Each requires five sections at the configured section size. Section traces and final matching gold values are retained separately. Twenty-four requested facts across three templates provide a diagnostic retention measure, not a general summarization benchmark or a test of unsupported-claim rate.

End-to-end times include the operations used by each method. Successful and failed chat attempts, token counters where returned, tool traces, and raw responses are retained. A one-second sampler records total device GPU memory and utilization, plus the summed resident memory of the benchmark process, explicitly selected runtime server, and their descendants. Summed RSS can double-count shared pages; total GPU memory includes the desktop. These observations are neither isolated model allocation nor energy measurements.

Each run stores dataset and corpus-file SHA-256 hashes, model digests, harness hash, options, and package versions. Resume is rejected if identity changes; a new run identifier is required. Failures are retained rather than silently removed. The accompanying builder's `-check` option verifies byte-identical synthetic artifacts. These measures support auditability without exposing private files.

TABLE II
COMPLETE-EVIDENCE SUCCESS ON 96 ANSWERABLE QUESTIONS

Method	CES@1	CES@3	CES@5	MRR@10
Lexical	86.46%	93.75%	100.00%	0.9948
Dense	68.75%	91.67%	94.79%	0.8755
Hybrid RRF	80.21%	94.79%	100.00%	0.9495

CES requires both sources for a two-document question. At five results, project-cluster bootstrap intervals are [100,100]% for lexical/hybrid and [91.67,97.92]% for dense. Degenerate empirical intervals do not establish population certainty.

VIII. RESULTS

A. Retrieval Effectiveness (RQ1)

All 39 documents were indexed without an extraction or embedding error, producing 141 chunks. Table II reports complete-evidence success. Lexical and hybrid retrieval both cover all required facts for 96/96 questions within five results; dense retrieval covers 91/96. The five dense failures occur among paraphrase questions, yielding 31/36 in that category. All three methods achieve complete coverage at ten results. This is evidence retrieval on the synthetic questions, not 100% answer accuracy.

First-result evidence-fact recall is 92.71%, 75.00%, and 86.46% for lexical, dense, and hybrid, respectively. Hybrid improves coverage over dense at five results, but adds no CES@5 gain over lexical and has lower MRR. The retained project names and repeated document structure favor exact lexical signals. Fig. 2 therefore supports retaining a lexical baseline rather than presuming that fusion is always superior. No public benchmark performance or general superiority is inferred.

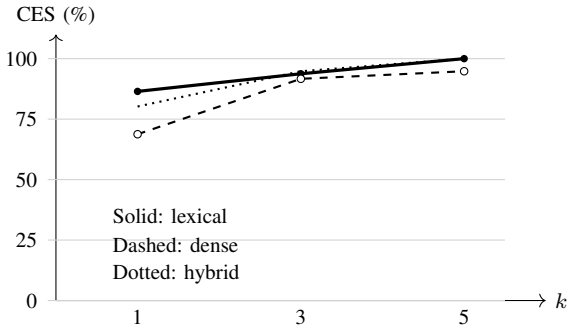


Fig. 2. Evidence coverage improves with the result budget. Fusion matches lexical CES@5 but does not improve its first-relevant-hit rank on this identifier-rich synthetic corpus.

TABLE III
EVIDENCE AVAILABILITY BEFORE PRIMARY GENERATION

Stage, positive questions only	Fact slots	Complete
Full returned source passages	32/32	24/24
First 1,000 characters/source	15/32	7/24
Messages retained by original fitter	0/32	0/24

Counts apply to both primary methods. Unknown-value questions have no gold fact slots. A complete question requires all of its one or two values.

B. Frozen-Baseline Answer Behavior (RQ2)

The frozen baseline produces a counterintuitive failure: both single-pass RAG and the original MCP agent match zero complete answers out of 24 answerable questions and have zero partial fact-value recall. Both return the requested absence phrase on all eight missing-value questions, but also on all 24 positive questions. No additional model-selected tool executes, no inference request errors, and each response uses one successful chat call. Thus 8/8 absence-phrase compliance is an always-abstain outcome, not evidence of calibrated missing-information handling.

The primary result is retained unchanged. The full retrieved sources cover all 32 required fact slots for the 24 generation questions. The failure is therefore not explained by absent indexed evidence. An offline reconstruction of the actual fitted messages identifies two distinct information-loss stages, shown in Table III. The first 1,000 characters of each passage cover only 15/32 fact slots and seven complete questions. The subsequent context fitter retains no document-evidence block for any of the 32 questions in either method.

The original fitting code estimates a raw content prefix from the remaining byte budget, appends a shortening marker, and then rechecks serialized JSON size. Escaped quotes and backslashes in already serialized evidence can increase that final size beyond the estimate. When the recheck fails, the block is omitted without another shrink attempt. For Q005, the single-pass remainder is 5,290 bytes, but its post-clip serialized message costs 5,354; the agent has 2,786 bytes after its schemas, but the message costs 2,806. Both blocks are consequently discarded. Low recorded prompt-token counts corroborate the missing input. The installed model template explicitly supports later system messages; the observed cause is the application's fitter, not a demonstrated inability of Qwen3 to use retrieved facts.

C. Post Hoc Exact-Cost Diagnostic

To test the identified mechanism, a separate evaluation adapter replaces only context-prefix fitting in memory. It uses binary search to choose the longest character prefix whose actual serialized JSON cost, including the shortening marker, fits the remaining budget. The original message priorities, pinned inputs, schema thresholds, passage order, 1,000-character passage prefixes, model, prompts, and condition order remain unchanged. Nine quote/backslash/Unicode cases across three context sizes, plus pinning, maximal-prefix, schema, and priority checks verify the bound without gold-dependent selection. This initial adapter did not modify the application or frozen primary results. The later production repair is evaluated separately below. Because both follow inspection of the same questions, they are development evidence, not independent confirmation.

All 64 diagnostic responses retain reference input. Single-pass AFVM becomes 7/24 (29.17%; project-cluster interval [12.50,50.00]%), while agent AFVM is 3/24 (12.50%; [0.00,25.00]%). Macro-averaged FVM is 37.50% and 27.08%, respectively. Exact-fitted initial contexts contain 15/32 gold value slots for single-pass RAG and 11/32 for the schema-bearing agent, covering seven and three complete questions. No model-selected additional tool executes. Thus the experiment shows a context-delivery correction, but no observed benefit from adaptive agent tool use on this set. Both conditions fail all eight lexical questions after prefix cropping; the single-pass method completes four cross-document and three paraphrase questions, and the agent completes three paraphrases.

Table IV also exposes the difference between reference presence and correctness. Diagnostic answers cover all gold document paths in 11/24 single-pass cases and 4/24 agent cases, despite fewer complete answers. One diagnostic agent response incorrectly treats Cobalt Ridge's internal review date as the date of an external security audit, which the documents never record. Seven of eight negative questions contain the requested absence phrase; the remaining case is a substantive date misattribution. Prefix truncation can remove the relation surrounding an otherwise familiar value, and citation indices alone do not detect that error.

Diagnostic median end-to-end times are 0.778 s for single-pass and 1.133 s for the MCP agent. The retained first single-pass case takes 36.17 s during an embedding-model reload; it is not discarded. Warm-up and prefix caching, the unbalanced order of negative cases, schema overhead, and MCP startup constrain timing interpretation. The shared frozen questions and nonzero-temperature single runs also prevent attributing every output difference solely to the fitter.

TABLE IV
FROZEN BASELINE, HISTORICAL DIAGNOSTIC, AND PRODUCTION-REPAIR REPLAY

Condition	AFVM ($n = 24$)	Macro FVM (%)	Absence phrase ($n = 8$)	Gold paths cited ($n = 24$)	Median (s)	95th percentile (s)
Frozen RAG	0/24	0.00	8/8	0/24	0.190	0.213
Frozen MCP agent	0/24	0.00	8/8	0/24	0.825	1.221
Exact-fit RAG (diagnostic)	7/24	37.50	8/8	11/24	0.778	3.018
Exact-fit MCP (diagnostic)	3/24	27.08	7/8	4/24	1.133	1.699
Repaired RAG (production)	7/24	37.50	8/8	9/24	0.686	1.255
Repaired MCP (production)	3/24	27.08	7/8	4/24	1.237	1.847

AFVM requires all requested values; FVM averages per-question alias coverage. Gold-path citation coverage is weaker than claim entailment and can occur on an incomplete answer. The historical diagnostic uses an in-memory adapter; production rows use the actual local repair. Both reuse inspected questions and are not held-out evaluation. All conditions use one chat call and no additional model-selected tool. Baseline timings measure mostly false abstentions.

D. Production Repair and Direct Replay

The subsequent repair changes production context fitting in both local platform copies. Accounting uses the serialized UTF-8 message cost, including escaping, metadata, and the shortening marker, while retaining pinned inputs and whole tool-call metadata. Nineteen added regression cases cover escaping, multilingual text, exact and tiny budgets, immutability, metadata, and the actual outgoing payload. Fourteen of the initial fifteen cases fail on the old fitter. Independent review also exercises 3,000 deterministic randomized budget cases; these checks support the implementation bound, not a tokenizer or semantic guarantee.

The separate production replay records initial reference evidence in 64/64 requests and verifies that every model call satisfies the configured serialized-input budget. It uses actual production code and MCP, with no diagnostic substitution and no request errors. AFVM is 7/24 for RAG and 3/24 for the agent, with project-cluster intervals [12.50,50.00]% and [0.00,25.00]%; macro FVM is 37.50% and 27.08%. Missing-value phrase compliance is 8/8 and 7/8. Each response uses one chat call, and no extra model-selected tool executes.

These counts match the historical diagnostic's AFVM, but do not make the runs interchangeable: production gold-path coverage is 9/24 for RAG and 4/24 for the agent, and timings differ (Table IV). The same inspected questions, model settings, and nonzero-temperature single runs constrain causal and generalization claims. The repair verifies evidence-block retention, while the separate 1,000-character passage prefix still loses facts. It does not establish complete context coverage, reliable semantic answers, or an adaptive-agent advantage.

E. Frozen Summary Retention (RQ3)

In the frozen baseline, all three long documents traverse actual MCP pagination and process all five extracted sections, yielding 15/15 section traces. Summarization has not been rerun after the production repair; these historical scores must not be presented as current summary validation. Each uses eleven model calls through map and recursive merge. Final outputs retain 3/8, 7/8, and 0/8 requested values, respectively, for 10/24 overall (41.67%). Table V separates this value retention from complete input visitation.

TABLE V
FROZEN FULL-DOCUMENT SUMMARY RESULTS (NOT RERUN AFTER REPAIR)

Dossier	Sections	Calls	Values	Time (s)
Morrow Annex	5/5	11	3/8	64.38
Oriole Basin	5/5	11	7/8	57.61
Tamarisk Span	5/5	11	0/8	56.75

The outputs also contain erroneous global claims that requested facts are absent from the original dossier. For example, the Oriole Basin response lists seven accepted values yet describes the document as incomplete. Alias coverage alone therefore overstates coherent factual adequacy. Bounded merging and the loss of information in partial summaries are plausible contributors; the primary run does not isolate their individual causal effects. The result directly rejects the assumption that reading every section guarantees a complete or correct summary.

F. Runtime and Functional Validation (RQ4)

In the frozen experiment, the 39-file/141-chunk index completes in 12.88 s. Median one-query embedding time is 13.60 ms. Median lexical and hybrid store ranking times are 4.02 and 4.76 ms on this small unchanged index, excluding query embedding and MCP startup. These values describe measured components and cannot be extrapolated to a large disk archive.

Frozen-baseline median end-to-end times are 0.190 s for single-pass RAG and 0.825 s for the MCP agent; their 95th percentiles are 0.213 and 1.221 s. Those responses are short abstentions with missing evidence, so the times are not representative latency for successful grounded answers. Full-document summaries take 56.75–64.38 s. Across 239 resource samples, the peak observed summed RSS is 8,258.89 MiB, and peak total-device GPU memory is 5,317 MiB. RSS may double-count shared pages and GPU totals include the desktop; neither is an isolated model requirement.

Functional evidence is separated by version and execution platform in Table VI. The repaired Linux suite passes 62 cases; the repaired Windows copy tested on Linux passes 99 with six native Windows checks skipped. Identical repaired inference

TABLE VI
CURRENT AND HISTORICAL FUNCTIONAL EVIDENCE

Platform	Interpreter / scope	Passed
Linux, repaired	Current suite, including context regressions	62/62
Windows copy	Current copy on Linux; 6 native checks skipped	99/99
Linux, frozen	Historical baseline suite	43/43
Windows Server	Historical native CPython 3.12.10	86/86
Windows Server	Historical native CPython 3.13.15	86/86

Local repair checks: 17 October 2026. Historical native Windows workflow: 6 October 2026. The repair has not been checked natively on Windows; Windows GPU inference and target Windows 11 laptop behavior remain unmeasured.

code and regression tests are present in both copies, and lint and whitespace checks pass. This is not new native Windows validation. The historical Windows workflow passed the same 86-case suite on each of CPython 3.12.10 and 3.13.15 on Windows Server 2025 [26], including Unicode paths, native boundaries, real MCP negotiation, packaging, and launcher cleanup. Its fake Ollama fixture performs no inference. The historical Linux suite passed 43 cases. Functional success and repaired payload bounds do not establish answer correctness or summary quality.

IX. DISCUSSION AND LIMITATIONS

A. Interpreting a Local Systems Prototype

The system's value is a locally controlled workflow with persistent project context and inspectable sources. It does not establish a novel retrieval model or universal superiority over existing assistants. A strong lexical result on identifier-rich notes would be compatible with prior retrieval literature [17]; a strong hybrid result would still be conditional on this corpus. RRF combines signals, but cannot guarantee that their fusion improves every query.

The agent's ability to request more evidence creates both an opportunity and a cost. Additional calls can recover omitted facts, while schema overhead and bounded prefix fitting still limit useful initial evidence. The production repair prevents the observed serialization overshoot but does not redesign passage selection. A small model can request unnecessary tools or answer before obtaining all required sources. The user-facing trace makes some of these decisions observable, but cannot expose every internal cause of an error. The measured comparison must therefore be read with the recorded tool activity and evidence coverage.

B. Coverage, Accuracy, and Traceability

Complete extracted-text storage, complete MCP pagination, complete section processing, and complete final factual coverage are different properties. Tests can establish the first two under specified inputs. A trace can establish that a processing loop visited all sections. Only an explicit output evaluation can address the last property, and alias matching covers only selected values. A short answer with all values can still contain an unsupported claim, while a faithful paraphrase outside the alias list can be penalized.

Likewise, showing a source card or an index such as [1] is weaker than proving that the cited passage entails a particular statement [20]. Source-number validity and document coverage are useful diagnostics, but should not be marketed as citation accuracy. A stronger study would annotate atomic claims, their supporting passages, contradictions, and justified abstentions with independent assessors.

C. Security, Cost, and Platform Limits

Read-only model tools reduce the effects of an instruction embedded in a document; they do not prove resistance to prompt injection [16]. The server itself offers mutations to authorized external clients, and the local OS account can read unencrypted extracted text. No adversarial red-team study, formal information-flow proof, packet-capture audit, or confidentiality certification is reported. Model/runtime installation also depends on external distributions and updates.

No paid inference API is required in the measured workflow. Nevertheless, the model files, duplicated extracted text, vector BLOBs, GPU runtime, and index consume disk space, and inference consumes power. No energy or total-cost-of-ownership measurement was made. A laptop's thermal policy, power mode, battery state, background applications, driver, and storage can alter timings. One device cannot establish a hardware-independent service-level guarantee.

The Windows edition preserves the Linux branch and uses native path and process APIs. Historical CI exercises native Windows Server, not the target Windows 11 GPU, and predates the local repair. The repaired Windows copy has only been checked on Linux with native checks skipped. Functional portability therefore has a narrower evidence base than cross-platform model performance. No Windows inference results are inferred from Linux GPU observations or from a fake-runtime launcher fixture.

D. External Validity and Future Work

LM-Synth is small, English, templated, and text-only. It excludes real personal task distribution, noisy extraction, OCR, long code dependencies, multilingual questions, conflicting document versions, and large-archive scale. Gold aliases and related templates introduce dependence and possible scoring biases. Cluster resampling represents uncertainty within twelve fictional families; it does not transform them into independent real users. Three summaries and eight absent-value questions are especially small samples. A single nonzero-temperature run does not characterize variation across random seeds. The production repair and its replay were developed after inspecting the original failure and reuse those questions; their successful payload checks and improved value scores are not independent or held-out confirmation.

Future evaluation should include consented real-world libraries, public heterogeneous retrieval tasks, multiple random seeds and devices, independent claim-level annotation, and calibrated abstention. Engineering work includes optional

OCR, stronger freshness verification, scalable vector indexing, protected storage and backup, summary merges that preserve fact identifiers, and user-controlled retention. Such features require new measurements and should not be described as implemented results.

X. Conclusion

LocalMind integrates local inference, persistent relational project memory, hybrid retrieval, and actual MCP communication on a constrained laptop. On LM-Synth v1, lexical and hybrid retrieval cover all 96 questions within five results, while the frozen generation baseline matches no complete answers because its fitter drops evidence. A subsequent local production repair fits actual serialized UTF-8 cost. Its separate direct replay retains initial references in 64/64 requests within budget, has no request errors, and matches 7/24 single-pass and 3/24 agent answers. These reused-question alias scores verify a repair and retain unresolved passage-prefix losses; they are not general semantic accuracy. Historical summaries visit every section but retain only 10/24 values and have not been rerun. Current functional checks pass 62 Linux and 99 Windows-copy-on-Linux cases, with native checks skipped for the latter. The repair is locally applied and unpublished; historical measurements remain intact. General accuracy, adversarial security, native Windows repair validation, and Windows laptop GPU performance require further evidence.

Appendix A
Reproduction and Artifact Map

To reproduce the frozen baseline, use Linux commit `ddbbbb7` and its recorded Python environment; do not substitute the repaired working tree for that historical source. The accompanying `research/evaluation` directory contains the corpus builder, frozen dataset, corpus specification, benchmark harness, and run-specific raw outputs. Run the builder's `-check` mode before evaluation. Keep only `corpus/` as the approved source root; do not index labels or answers.

Start the downloaded Ollama runtime on a free loopback port with cloud inference disabled, one parallel request, the two exact installed model artifacts, and the existing model directory. Pass that server's PID to the harness for RSS attribution. The default experiment command is `python research/evaluation/run_benchmark.py -phase all -runtime-pid PID`; a changed corpus, harness, or model requires a fresh `-run-id`. The supplied protocol files identify the actual run rather than promising bit-identical stochastic generation. The companion evaluation README gives complete launch commands and explains metric definitions.

The production-repair evidence is separate under `research/evaluation/results/2026-10-17-production-context-fix-v1/`; `repair-verification.json` records source hashes, direct-production execution, budget checks, and results. The matching summary is `output/verification/context-fitting-fix.json`. The repaired `copilot/ollama.py` has SHA-256

```
1db1e79b7eac3dba77b2afaa2e852db8
47b7ae2169d7f92327b750dbaa4cdc25
```

The two printed lines concatenate to one digest. This working-tree repair is not a published commit; both local platform copies match. Frozen corpus, baseline, and diagnostic outputs remain unchanged. A fresh process is needed to load the repair; no reindexing or model change is required. Reproduction of the production replay must use the recorded repaired file rather than claim that the published baseline already contains it.

For application use, add approved knowledge folders, index them, and inspect source cards alongside answers. Complete extracted-text reads follow pagination until `next_offset` is null. Saving memory and drafting a plan require explicit actions. Network service exposure, hosted external MCP clients, and private-file publication are outside this reproduction procedure.

Acknowledgment

The authors thank Dr. Sonali Kothari for her project guidance and supervision.

AI assistance was used in drafting this manuscript, evaluation code, and qualitative error inspection. Quantitative results derive from executed local experiments and deterministic scorers; no language model scored the benchmark.

References

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, doi: 10.1145/3359591.3359737. [Online]. Available: https://martin.kleppmann.com/papers/local-first.pdf

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. arXiv:2005.11401. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023, revised 2024. [Online]. Available: https://arxiv.org/abs/2310.08560

[4] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19724–19731, doi: 10.1609/aaai.v38i17.29946. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/29946

[5] Model Context Protocol contributors, "Model Context Protocol specification," protocol revision 2025-11-25, 2025. Accessed: Oct. 17, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[6] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410. [Online]. Available: https://aclanthology.org/D19-1410/

[8] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019. [Online]. Available: https://www.nowpublishers.com/article/Details/INR-019

[9] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114. [Online]. Available: https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf

[10] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023. arXiv:2210.03629. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[11] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2302.04761. [Online]. Available: https://arxiv.org/abs/2302.04761

[12] W. Wang *et al.*, "Augmenting language models with long-term memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2306.07174. [Online]. Available: https://arxiv.org/abs/2306.07174

[13] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," arXiv preprint arXiv:2304.03442, 2023. [Online]. Available: https://arxiv.org/abs/2304.03442

[14] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2303.11366. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[15] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[16] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv preprint arXiv:2302.12173, 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[17] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021. arXiv:2104.08663. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/65b9eea6e1cc6bb9f0cd2a47751a186f-Abstract-round2.html

[18] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2023, pp. 2014–2037, doi: 10.18653/v1/2023.eacl-main.148. [Online]. Available: https://aclanthology.org/2023.eacl-main.148/

[19] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158, doi: 10.18653/v1/2024.eacl-demo.16. [Online]. Available: https://aclanthology.org/2024.eacl-demo.16/

[20] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488, doi: 10.18653/v1/2023.emnlp-main.398. [Online]. Available: https://aclanthology.org/2023.emnlp-main.398/

[21] Ollama, "FAQ," official software documentation. Accessed: Oct. 17, 2026. [Online]. Available: https://docs.ollama.com/faq

[22] TrijalBurman, "LocalMind local knowledge copilot—Linux edition," source code, commit `ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8`, Oct. 3, 2026. [Online]. Available: https://github.com/TrijalBurman/MCP-Server/tree/ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8

[23] SQLite developers, "SQLite FTS5 extension," official implementation documentation. Accessed: Oct. 17, 2026. [Online]. Available: https://sqlite.org/fts5.html

[24] A. Yang *et al.*, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[25] H. Schechter Vera *et al.*, "EmbeddingGemma: Powerful and lightweight text representations," arXiv preprint arXiv:2509.20354, 2025. [Online]. Available: https://arxiv.org/abs/2509.20354

[26] TrijalBurman, "Native Windows validation workflow, run 37470160711," GitHub Actions, commit `0e0bbf6f944bc6f2534f882c858ca4cdd22e6443`, Oct. 6, 2026. Native Windows Server 2025 jobs using Python 3.12.10 and 3.13.15. [Online]. Available: https://github.com/TrijalBurman/MCP-Server/actions/runs/37470160711